Latest updates: https://dl.acm.org/doi/10.1145/3746252.3761551

RESEARCH-ARTICLE

Zipf-Gramming: Scaling Byte N-Grams Up to Production Sized Malware Corpora

EDWARD RAFF, CrowdStrike, Inc., Sunnyvale, CA, United States

RYAN R CURTIN, Booz Allen Hamilton, Inc., McLean, VA, United States

DEREK EVERETT, Booz Allen Hamilton, Inc., McLean, VA, United States

ROBERT J JOYCE, Booz Allen Hamilton, Inc., McLean, VA, United States

JAMES HOLT, Laboratory for Physical Sciences, College Park, MD, United States

Open Access Support provided by:

Booz Allen Hamilton, Inc.

CrowdStrike, Inc.

Laboratory for Physical Sciences

PDF Download
3746252.3761551.pdf
11 February 2026
Total Citations: 0
Total Downloads: 106

Published: 10 November 2025

Citation in BibTeX format

CIKM '25: The 34th ACM International Conference on Information and Knowledge Management
November 10 - 14, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGWEB
SIGIR

CIKM '25: Proceedings of the 34th ACM International Conference on Information and Knowledge Management (November 2025)
https://doi.org/10.1145/3746252.3761551
ISBN: 9798400720406

Zipf-Gramming: Scaling Byte N-Grams Up to Production Sized Malware Corpora

Edward Raff
edward.raff@crowdstrike.com
CrowdStrike
Austin, USA

Ryan R. Curtin
Derek Everett
Robert J. Joyce
curtin_ryan@bah.com
everett_derek@bah.com
joyce_robert2@bah.com
Booz Allen Hamilton
McLean, USA

James Holt
Laboratory for Physical Sciences
College Park, USA
holt@lps.umd.edu

Abstract

A classifier using byte n-grams as features is the only approach we have found fast enough to meet requirements in size (sub 2 MB), speed (multiple GB/s), and latency (sub 10 ms) for deployment in numerous malware detection scenarios. However, we've consistently found that 6-8 grams achieve the best accuracy on our production deployments but have been unable to deploy regularly updated models due to the high cost of finding the top-k most frequent n-grams over terabytes of executable programs. Because the Zipfian distribution well models the distribution of n-grams, we exploit its properties to develop a new top-k n-gram extractor that is up to 35× faster than the previous best alternative. Using our new Zipf-Gramming algorithm, we are able to scale up our production training set and obtain up to 30% improvement in AUC at detecting new malware. We show theoretically and empirically that our approach will select the top-k items with little error and the interplay between theory and engineering required to achieve these results.

CCS Concepts

• **Security and privacy → Malware and its mitigation**.

Keywords

malware detection; fast feature extraction

ACM Reference Format:
Edward Raff, Ryan R. Curtin, Derek Everett, Robert J. Joyce, and James Holt. 2025. Zipf-Gramming: Scaling Byte N-Grams Up to Production Sized Malware Corpora. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25), November 10–14, 2025, Seoul, Republic of Korea.* ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/3746252.3761551

1 Introduction

Malware detection and analysis involves numerous stages and pipelines due to the wide range of costs and risks involved. Our team developed and deployed a byte n-gram-based model for malware detection that has been used for several years within the U.S. Federal government [13]. The high-level approach is simple: extract n-grams from a large corpus of malware and benign programs, select the top-k most frequent grams as an initial feature set, and then use an L_1 or Elastic-Net penalized model to down-select the features and produce the classifier. This model's simplicity is the source of its value in production, coming with two specific mission-critical benefits: **(1)** The model has no required parser dependencies and cannot "fail" in parsing any individual file. **(2)** The feature extraction and classification of files is so small and fast that it can be done "at line speed", where the bottleneck is in getting the data into a computer system, rather than in computational effort.

The first benefit is needed for reliability in production; the tool can be safely deployed on a variety of systems that are of varying ages, software platforms, and language support. The inference code can be written in a few lines of C, Python, Java, etc., and can thus support a wide breadth of operational environments that occur within the U.S. government. The second benefit is critical in the application, as the volume of data that requires analysis is too great for any kind of parser; even simple disassembly parsers are too expensive, let alone more complex types of "lifting" like decompilation. Because byte n-grams can be extracted and evaluated against a linear model quickly, the model can be used as a first-pass filter to identify a smaller subset of files that need further analysis. This could be in the form of a more complex model, algorithm, feature extractor, or professional analyst for review. A wide array of needs and scenarios exist [8, 18–20, 25, 32, 48, 52], from simple "prevent infections" to forensic investigations to "find any possible malware", creating a range of FPR/TPR thresholds, for which we recommend each downstream user to adjust the threshold of the decision to their operational needs.

Our issue is in the process of extracting the top-k most frequent n-grams itself, which has become a computational burden preventing timely and effective model updates – a requirement for malware detection systems as malware evolves as an adversarial process. To resolve this, we develop an improved algorithm for extracting the top-k n-grams under the assumption they come from a power law distribution, specifically the Zipfian distribution, to safely and stochastically avoid work while arriving at a solution of equal quality.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CIKM '25, Seoul, Republic of Korea
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2040-6/2025/11
https://doi.org/10.1145/3746252.3761551

2 Related Work

Machine learning for malware is challenging for many reasons, including concept drift and the need to update models frequently [40, 58]. This has been a challenge for us to update this model as we were unable to increase the size of the training set and still provide timely updates. In our production model, we use multiple evaluation splits based on time to mitigate this and better determine the utility in deployment [34] so that we can have a more accurate estimation of real-world performance. Our use of byte-based features has been widely studied as the exclusive feature representation, but always on smaller datasets [4, 10, 12, 14, 15, 42, 57, 59, 61], and byte-based features have still been valuable to production models that perform parsing or feature extraction [53] or improve Transformer-based approaches in industry [49]. Popular perception is that AntiVirus (AV) systems do not use such simple methods due to advertisement of their ML capabilities, but reverse engineering shows that they still depend extensively on simpler approaches like signatures [5], of which byte n-grams are a major subset [11, 22, 43, 46]. The goal of this article is not to design a complete Anti-Virus system but to improve our faster model used to pre-filter files as the only viable option to getting some kind of result on all production data [30, 31].

As we look at byte n-grams, it is notable that power laws naturally arise in text [23, 35, 50] citation rates [45] code repositories [24], where there are a few items with high frequency and many items with exceptionally low frequency. Zipf's law or the Zipfian distribution is frequently relevant when the number of items is finite [29]. This is useful empirically and theoretically, as it eliminates the challenge of fitting the most correct power law given that other popular options are either continuous or for infinite support [6], which means we can, by the process of elimination, focus only on the Zipf distribution. By focusing on the worst-case Zipf, i.e., the parameterization with the weakest power law, we eliminate the need to estimate any parameters to obtain useful theory. Although the power-law nature of n-grams has been recognized, it has not been leveraged in feature extraction code. Relevant works in n-gramming have found it still a competitive baseline with deep-learning when properly tuned (that is much faster, a key need in our case) [54]. N-grams are also still critical in certain domains such as bioinformatics [62], where our 8-grams over a byte (256 value) alphabet could be adapted to support 32-grams over the 4 value alphabet in genome sequencing (or 64-grams if we used 128-bit long longs). Notably, finding the top-k most frequent n-grams (also called "k-mers" in bioinformatics) is itself a common need for n-grams in the range $[20, 60]$ [51]. While still important to bioinformatics, most popular tools perform chunking or caching to reduce in-memory use, but still naïvely count n-grams and filter after [47]. Though our approach is relevant to the task, k-mers also have a "quality score" for each item in the sequence that is beyond the scope of our work, where bytes are known and exact.

3 Fast Top-k n-Gramming

The models we want to train are byte n-grams, so there are 256^n possible n-grams. We have found that $n \in [6, 8]$ usually performs best, which means there are more possible n-grams than bytes in our training data to observe. A model that has any predictive utility must use n-grams that re-occur, and so selecting the top-k most frequent n-grams is a simple and effective initial step to model construction (i.e., a feature seen only once is improbable in being a helpful feature). Standard ML libraries that support n-grams do this in a naïve way: count all observed n-grams, and then select the top-k most frequent n-grams. This is manageable for $n \in [1, 3]$ as is common in text applications in moderately sized corpora. For our malware datasets that require larger n and are up to 32 terabytes in size, this is a computationally infeasible approach.

Instead, we are concerned with directly estimating the top-k items without holding an unbounded amount of additional memory, because most items are infrequent and observed ≤ 10 times. The canonical algorithm for finding the top-k most frequent items in a stream is the SpaceSaving algorithm [28]. While theoretically beautiful, this data structure achieves low throughput on modern hardware, making it a non-option. It was instead used as a component in the KiloGrams algorithm [37–39], which performed a two-pass process: **(1)** Estimate the top-k most frequent hashes of n-grams, as the hashes can be enumerated in constant memory. **(2)** On a second pass, use the Space-Saving algorithm only on the white-listed hashes from the first pass, reducing the total number of Space-Saving updates.

The top-k hash strategy is possible only because of the power-law distribution that n-grams tend to follow, as referenced in § 2. Because we have a finite alphabet, the Zipfian distribution (or "Zipf" for short) [63] is assumed for the distribution of n-grams. The top-k hashes will occur with much greater frequency than the lower-ranked/less-frequent items and so make an accurate correspondence to the true top-k items. While an improvement, we find that the KiloGramming algorithm is still too slow due to the large $\approx 2^{31}$ sized table it must create. We will improve upon the KiloGramming algorithm with our Zipf-Gramming approach.

Our Zipf-Gramming approach is shown in Algorithm 1, where blue and green lines correspond to our improvements over the original KiloGramming algorithm. If removed, the remaining black lines constitute the original KiloGramming approach. The key issue with KiloGrams is that updating the large hash table in the first pass (line 10) and the SpaceSaving data structure in the second pass (line 24) is prohibitively slow. Our strategy then is to avoid performing these updates as much as possible and do just enough updates to obtain a reasonable estimate of the true top-k items. In blue, we take advantage of the Zipf distribution to stochastically avoid counting n-grams in a manner that allows us to reach the same top-k items with high fidelity. In green, we create a smaller write/evict cache to act as a buffer to accumulate updates before pushing them to the larger data structures. These algorithmic improvements, combined with careful software engineering to avoid as many even mildly expensive operations as possible, combine to produce the first top-k n-gram algorithm that can process large datasets at gigabit speeds and greater for large datasets.

In the remainder of this section, we will first detail more explicitly and empirically why the KiloGramming approach is slow and how the hash-table results in worst-case hardware characteristics in § 3.1. Then, we will theoretically justify our stochastic counting approach in § 3.2. Because the write/evict cache is sensitive to implementation details, its proofs and discussion will occur in § 4.

Algorithm 1 Zipf-Gramming, blue and green are additions from the original Kilo-Gram algorithm

Require: Bucket size B, rolling hash function $h(\cdot)$, corpus of C documents, and desired number of frequent items k.

```
B ← new integer array of size |B|
Create Cache C of size |C| = k
Determine p_skip, the probability of skipping the given token.
for all documents x ∈ C do                ▷ O(L) for L total n-grams
    for n-gram g ∈ x do
        if p_skip < p ~ U(0,1) then
            Skip processing this n-gram.
        q' ← h(g) mod B
        if q' ≠ Key(C[q']) then           ▷ Check if g is in the cache
            B[q'] ← B[q'] + Value(C[q'])
            Key(C[q']) ← q'
            Value(C[q']) ← 0
        Value(C[q']) ← Value(C[q']) + 1
B_k ← QuickSelect(B, k)
S ← new Space Saving structure with B_S buckets.
Refine p_skip, the probability of skipping the given token.
for all documents x ∈ C do                ▷ Second pass over data
    for n-gram g ∈ x do
        if p_skip < p ~ U(0,1) then
            Skip processing this n-gram.
        q' ← h(g) mod B
        if q' ∈ B_k then
            if q' ≠ Key(C[q']) then       ▷ Check if g is in the cache
                Insert Key(C[q']) into S Value(C[q']) times
                Key(C[q']) ← g
                Value(C[q']) ← 0
            Value(C[q']) ← Value(C[q']) + 1
    return top-k entries from S
```

3.1 Bottlenecks of the KiloGramming

Although the *KiloGramming* algorithm produces a significant speedup compared to the naïve approach of maintaining a list of n-grams with counts, it suffers from some significant bottlenecks when implemented on real-world hardware. Primarily, the bottleneck is associated with B, the array used in the first pass that holds the counts for each hash value. Because the bucket size B is typically chosen to be a large value, such as $2^{31} - 19$, this means the amount of RAM required to hold B is 8 GB if 32-bit integers are used, and 16 GB if 64-bit integers are used. For modern systems, this is not an onerous requirement: RAM sizes are often 64 GB and larger.

But the memory access patterns of the *KiloGramming* algorithm are highly problematic: because each n-gram is hashed to a bin in B uniformly randomly, each successive update to B requires accessing a random location in B. This is a worst-case situation for memory access, as the hardware prefetcher cannot predict the next access location. As a result, the memory access request can only be issued once the hash value $h(g)$ is known, causing significant latency for each individual update.

Worse yet, because B is so large, it spans many pages of virtual memory. Before a memory access request can succeed, the virtual page must be mapped to physical memory. Virtual page mappings are cached by the translation lookaside buffer (TLB). If a memory access request corresponds to a virtual page that is *not* cached in the TLB—which is going to be virtually every lookup in B because we are accessing B fully randomly—a walk of the page table must occur. Each of these walks can be very expensive.

So, overall, the memory access pattern required by the *KiloGramming* algorithm is basically worst-case: random accesses in memory mean that hardware prefetchers are ineffective; the huge size of B means that many pages of memory are required, meaning constant TLB misses and page walks. This causes *KiloGramming*'s main bottleneck to be updates to B instead of reading the underlying data from disk. It is easy to demonstrate the significant effect that these memory access patterns have. Table 1 presents a simple example inspired by Drepper [7]: we generate an array of a few different sizes, and increment elements sequentially as well as randomly. The *KiloGramming* situation—random access, largest array size—runs at ~ 100 **MB/s**! This is many orders of magnitude away from peak memory bandwidth and is even a couple orders of magnitude away from typical modern disk speeds.

Size of array	Sequential	Random
4 kB	27.1 GB/s	0.56 GB/s
2 MB	32.5 GB/s	0.54 GB/s
8 GB	10.0 GB/s	**0.10 GB/s**

Table 1: At the sizes required for *KiloGramming*, the penalty is orders of magnitude!

Having now identified the bottleneck, we present our simple solution: just access B less often. It turns out that we can do this probabilistically without affecting the quality of the results.

3.2 Zipf Counting

We assume our n-grams follow a power law distribution, which is expected and previously observed when using n-grams [23, 24, 35, 45, 50]. In particular, we use the Zipf distribution, over an alphabet of Σ possible items, with the probability mass function $\mathrm{PMF}_{\mathrm{Zipf}}(x) = \frac{x^{-\rho-1}}{H_{\Sigma}^{(\rho+1)}}$ for $1 \leq x \leq \Sigma$ where $H_{\Sigma}^{\rho} = \sum_{i=1}^{\Sigma} \frac{1}{i^{\rho}}$ is the harmonic number. We will use $\rho = 0$ in our analysis as the worst-case possible distribution because it maximizes the weight that is in the tail.

Rather than process all L n-grams of a sequence, we want to subsample a fraction of only $N < L$ items to count, as this would save significant computational effort. Our interest then is in bounding the probability of making an error in selecting the top-k items when we sub-sample the distribution of n-grams.

Tackling the issue by molding it as a binomial distribution is unsatisfying because the value $\mathbb{P}(Z \leq 0)$ which corresponds to the probability of k_l getting a higher total count thank k_h (i.e., probability of an error) is loosely bounded and hard to calculate. Further, the more ideal result is to obtain the CDF for the ordered statistic of all $\min(|\Sigma|, L) - k_h$ distributions for all $k' < k_h$, so that

we can quantify the likelihood of *any* lower ranked item receiving a count greater than b_h.

We provide positive evidence for both of these questions in this section by leveraging the Gaussian approximation to the binomial distribution, $B(N,p) \approx \mathcal{N}(Np, \sqrt{Np(1-p)})$ which is generally accurate so long as $Np \geq 5$ [33, 44]. In our analysis, this is so that we have a more pliable distribution to analyze, so that we may better understand the reasonableness of our top-k sampling approach (under the caution of using an approximate distribution). Thus, we replace $b_k \sim \mathcal{N}(Np_k, \sqrt{Np(1-p_k)})$ where $p_k = \text{PMF}(\text{Zipf}, k_h))$.

The more challenging question is to bound the likelihood of a range of $k' \geq k_l$ receiving too high a count. Essentially, To bound the likelihood of a range of $k' \geq k_l$ receiving too high a count, let $Z^{k_h}_{k'}$ be defined by the distribution:

$$\mathcal{N}\left(Np_{k_h} - Np_{k'}, \sqrt{N(1-p_{k_h})p_{k_h} + N(1-p_{k'})p_{k'}}\right).$$

What we wish to calculate is $\mathbb{P}(\min\left[Z^{k_h}_{k_l}, Z^{k_h}_{k_l+1}, \ldots, Z^{k_h}_{\min(N,\Sigma)}\right] \leq 0)$, which corresponds to an ordered statistic over the Zs. Evaluating $\mathbb{P}(\min\left[Z^{k_h}_{k_l}, Z^{k_h}_{k_l+1}, \ldots, Z^{k_h}_{\min(N,\Sigma)}\right] \leq 0)$ is an ordered statistic for non-identical distributions, which is notoriously challenging [3]. Our approach is to upper-bound this distribution by replacing all $Z^{k_h}_{k_l+c}$ with $Z^{k_h}_{k_l}$, which is pessimistic but useful in judging a range of gaps, yielding 1.

Theorem 1. *The expected difference in observations between h_k and the T items after the h_l'th lower ranked item is*

$$\geq -\sqrt{2}\sqrt{\log(T)}\sqrt{\frac{N\left(1-\frac{1}{h_k H_\Sigma}\right)}{h_k H_\Sigma} + \frac{N\left(1-\frac{1}{h_l H_\Sigma}\right)}{h_l H_\Sigma}} + \frac{N}{h_k H_\Sigma} - \frac{N}{h_l H_\Sigma}$$

Proof. The sketch of the proof, due to space, is as follows: use the binomial approximation and i.i.d. distribution of counts to use Jensen's inequality for the ordered statistic (maximum) over Gaussians. The min/max are the same due to symmetry (with a sign change). The minimum is applied as a lower bound due to linearity of expectation, giving $\mathbb{E}[\min\left[Z^{k_h}_{k_l}, Z^{k_h}_{k_l+1}, \ldots, Z^{k_h}_{T}\right]] \leq -\mathbb{E}[\mathcal{X}_{\max}] \geq -\sqrt{\mathbb{V}[Z^{k_h}_{k_l}]}\sqrt{2\log T} + \mathbb{E}[Z^{k_h}_{k_l}]$. Substituting the mean and variance for the Gaussian approximation of the binomial recovers the proof. □

We use 1 to demonstrate that even over an order of magnitude of lower-ranked items, we still expect, on average, the approach of sub-sampling Zipfian distribution to accurately select from the top-k items. In Figure 1, we plot this bound normalized by the least frequent but desired item k_h's expected count (i.e., how much excess buffer in counts occurs). We see that this value is positive, and the buffer increases as we choose larger thresholds k_h.

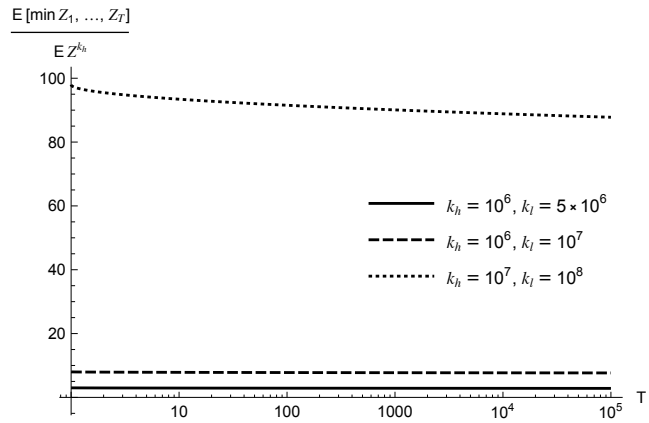

Figure 1: The y-axis shows the ratio of excess expected counts to select the k_h'th item over the preceding cumulative T items (x-axis). Using relevant real-world values of $\Sigma = 256^6$ and $n = 10^{12}$, we see that we always expect a positive value (and thus no errors on *average*).

4 Efficient Implementation

As a motivating example of the need to be careful in implementation, consider that a modern high-end 5 GHz CPU can perform $\approx 43,000$ million instructions per second, which is (roughly) 43 instructions per nanosecond[1]. A top-of-the-line NVMe SSD can obtain a theoretical maximum throughput of ≈ 7 GB/s, or 7 bytes per nanosecond. While both numbers are theoretical maximums that cannot usually be achieved, it puts into context that at optimal compute efficiency, just moving bytes constitutes 16% of all available compute to obtain maximum data transfer rates. As we illustrated in § 3.1, the compute portion of the original KiloGram process is exhibiting the worst-case behavior. Thus, any complex data structure update (like SpaceSaving) is prohibitively expensive and needs to be minimized, and any non-update states must be highly efficient to make up for the lost time on updates. This is especially important in the first pass of the algorithm when there is no whitelist to further reduce the data structure update rate.

We rely heavily on bit-twiddling operations like AND, OR, and XOR that have a latency of 1 cycle in modern hardware, and integer multiplications are usually only 3-5 cycles. But even a modulo operation (e.g., `hash % range`) to index into a table can be too expensive at 15-71 cycles. Floating point operations can vary from 9-400 cycles, and so must be avoided at all costs [9].

In this section, we will review low-level details and implementation decisions that we found were required to reach high data throughput. This includes faking a "rolling" hash function given that we are skipping most bytes, how to avoid floating point math for determining the next byte to evaluate, and the choice behind and proof of our write/evict cache strategy.

4.1 Rolling Hash Function $h(\cdot)$

The original KiloGram's implementation used the Rabin Karp rolling hash function (RKH). Given a sequence of $a_1, \ldots, a_n$ tokens $h(\cdot) = a_1 + a_2 B + \ldots + a_n B^n$ where B is a (generally prime) constant. When one wants to shift or "roll" the has function over by one token to a_{n+1}, you can calculate this in constant time as $B \cdot h(\cdot) + a_{n+1} - B^n \cdot a_1$. By pre-calculating B^n, this can be updated using only two integer multiplication and two integer additions.

[1] e.g., see https://hothardware.com/reviews/intel-core-i9-11900k-core-i5-11600k-rocket-lake-s-review

However, we found that this hash function is prohibitively slow in practice due to low computational throughput. This has been observed by others in the HPC and software engineering space, where the naïve RKH can only run at 0.75 GB/s, absent any other operations. This is far below the 7 GB/s theoretical IO throughput of our hardware and optimistic as it excludes any work done around/with the hash values themselves [21, 55]. For example, even though we will skip some number of bytes with probability p_{skip}, the RKH hash requires us to still calculate the hash value for skipped tokens because the hash for the t'th token in a sequence is dependent on the $n-1$ previous tokens.

Instead, we use a 64-bit integer to keep a running estimate of the current "key" n-gram. This can be maintained by a simple bit shift and logical and operation at each new input token; $key = key \ll l|a_{n+1}$ is sufficient to calculate this with highly efficient bitwise operations. Only when we are at a location selected to be counted do we calculate a hash value from the current n-gram key. For the hash function, we use the SplitMix hash function [56], which has good uniformity of distribution, and requires only 6 bit-twiddles and two integer multiplications. Thus, it is of comparable cost to RKH, but only when we need a hash with better statistical distribution.

4.2 Skipping Tokens & Geometric Distribution

In lines 6-7 and 19-20 of Algorithm 1, we select with probability p_{skip} to skip the current item for processing. This optimization is critical for avoiding cache misses when indexing into the large matrix on line 10, and the high computational cost of the SpaceSaving algorithm on line 24. However, running any Pseudo-Random Number Generator (PRNG) on every byte sequence is a non-trivial computational cost of its own.

In an ideal world, we would know how many bytes s to skip forward to the next n-gram, which would allow us to skip to the next $s-n+1$ bytes so that we can ensure the key storing the current gram g is correctly populated (i.e., you must record the $n-1$ tokens to know the current n-gram). Naïvely, we could set $s = \lfloor 1/p_{\text{skip}} \rceil$, but this would result in a biased distribution of byte values. This is especially problematic for malware, where content is often aligned to various power-of-two multiples due to a variety of technical reasons.

Theorem 2. *For an arbitrary threshold $c \in (0,1)$, the inverse CDF of s can be pre-calculated exhaustively in $\Theta\left(\frac{1}{1-p_{skip}}\right)$ values.*

Proof. The number of trials to success follows the Geometric distribution [2]. Solving the PDF for the smallest input that changes the output by a unit of 1 gives the size of a cache $|S| = \frac{x \cdot p_{\text{skip}} - x - 1}{p_{\text{skip}} - 1}$ to enumerate all values. Since x is not known, we take the CDF(x, p_{skip}) = c of the Geometric and solve for x, to get a cache size $|S| = \frac{\log(1-c)}{\log(p_{\text{skip}})} + \frac{1}{1-p_{\text{skip}}} - \frac{1}{2}$, which grows at a rate of rate of $\Theta\left(\frac{1}{1-p_{skip}}\right)$ □

2's result, when we round the pre-calculate value cache size $|S|$ up to the next power of two, means we can cheaply sample the next 32-bit integer and shift right by $32 - \log_2(|S|)$ (special assembly functions exist to compute this in 1 cycle) bits to get the next value of $s \leftarrow S[\cdot]$. Doing so, we can represent 99.9% of the Geometric distribution range for $p_{\text{skip}} \leq 99.9\%$ (i.e., all practical values in our experiments) within a cache of just 32 KB of cache (easily fits into L1 cache). The 0. 1% not represented is not a concern, as it is the tail of the distribution going toward ∞, and capping the 0.1% can only cause us to over-sample a (slightly) larger value of N.

4.3 Simple Write/Evict Cache

Our write/evict cache in Algorithm 1 is a simple data structure where we wish to accumulate the frequent updates into larger increments before reaching out to the main data structures. One may intuitively think that a Least Recently Used (LRU) or other kind of data structure would be appropriate for this task, but the overhead of these data structures is too great given the low overhead we can tolerate. Instead, a simple strategy of indexing into a small table and, on collision, evicting and replacing the previous value is used. By making this table the nearest power of two we can index into it using the rolling hash function quickly (avoiding a long latency modulo operation). One may hypothesize that such a naïve strategy would result in no benefit as the majority of items that occur less than once would overwhelm the cache, but we prove this strategy results in a log-linear reduction in main memory accesses (i.e., to the large hash-table or the SpaceSaving data structure).

Theorem 3. *A cache of size $|C|$ reduces the number of writes to main memory by a factor of $\Theta(|C| \log |C|)$.*

Proof. By the pigeonhole principle one can compute an eviction probability as $p_{\text{evict-x}} = \frac{f(x)}{f(x)+(1-f(x))/|C|}$, substituting the Zipf's PMF for $f(x)$ to obtain $\frac{|C|}{|C|+xH_{\Sigma}-1}$, which is used in the Geometric distribution to get the per-entry hit rate, multiplied by $|C|$ entries, and take the limit divided by $|C| \log |C|$ as $|C| \to \infty$ to prove $\Theta(|C| \log |C|)$. □

5 Results

We now review the results of our method. First, we will establish that Zipf-Gramming has much higher computational throughput, especially compared to standard tools like Scikit-Learn. Second, we will establish that Zipf-Gramming results in similar accuracy on a variety of different file types of moderately sized corpora. Third, we will establish that Zipf-Gramming empirically captures the majority of true top-k features on a moderately sized corpus. Finally, only Zipf-Gramming can run on our production scale experiments, where we document significant gains in AUC and up to 76% absolute improvement in identifying challenging malware in our production environment.

Our results will be reported in AUC as our model is used at multiple different TPR vs. FPR thresholds, and we advise each downstream deployment to select a threshold applicable to their specific needs. For example, generic malware detection tasks will prefer a low FPR to avoid user issues. However, forensic investigations and other government tasks[2] often need maximal TPR, even when large FPR occur. Similarly, this is used as a component in larger systems as a faster pre-filter. This study is concerned only with the quality of this component, which has wide versatility in deployment.

[2] https://afresearchlab.com/technology/successstories/cross-domain-solutions-101/

5.1 Computational Throughput

In our first experiments, we examine the computational throughput of Zipf-Gramming and comparable baselines. In fairness to all prior approaches, we modify them as necessary to maximize their similarity to Zipf-Gramming only if the modification improves the MB/s of the prior approach. For example, the Kilo-Gramming implementation counts an n-gram only *once per file* , which causes a $\approx 10\times$ slowdown as it must allocate a large dictionary, often larger in used memory than the file being processed due to overhead.

Our primary results are shown in Figure 2, where we see that computing the top 100,000 *n*grams for our Zipf-Gramming algorithm is 492× faster than Scikit-Learn when Scikit-Learn is given the benefit of using just 3-grams, as it could not successfully complete a run at 8-grams. These results were obtained running on a M3 Macbook Pro with 96 GB of RAM. Our method is the only one that can process larger corpora and continues to increase in efficiency as we do so. At $4.6 \cdot 10^{12}$ bytes of input, Zipf-Gramming reaches 2,762 MB/s, which would be 35.3× faster than Kilo-Grams if we could wait long enough for it to finish running. Scikit-Learn CountVectorizer was tested using a 2 GB file for 6-grams. Using IO stats we observed bursts up to 63 KB/s, with multiple gaps of 0 KB/s, but the program crashed after several hours of computation.

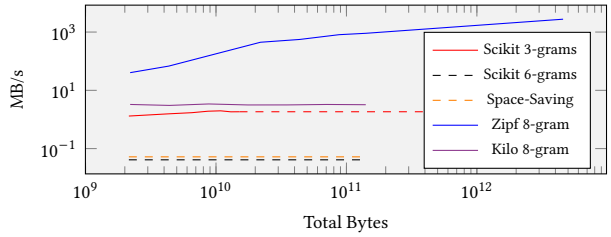

Figure 2: Our Zipf-Gramming is orders of magnitude faster (y-axis) than alternatives, and performance improves as dataset size (x-axis) increases. Dashed lines are not run to completion but measured with IO profiling tools due to exceeding a 1-hour timeout.

It is worth noting that in the Kilo and Zipf-gramming algorithms, two passes over the dataset are made, and we are measuring the MB/s over the entire runtime of the program — not just the IO throughput per step. This means our Zipf-Gramming is reaching 5,524 MB/s IO throughput, and the maximum measured IO throughput for this machine when reading data to /DEV/NULL is 5777.7 MB/s, indicating we are at 95.61% of maximal practical efficiency and 73.65% of the theoretical maximum of 7,500 MB/s over the PCIe 4.0 interface used by our drive.

This also makes our approach particularly efficacious for any researcher developing on a local machine. Most NVMe connectors receive only four PCIe lanes, which would have a maximum theoretical throughput of 875 MB/s, and consumer SSDs connected via a SATA have a maximum theoretical throughput of 750 MB/s[3]. Our method handily surpasses what most individuals' workstations will be able to achieve. While developing a single-pass algorithm that can obtain higher theoretical runtime is desirable, only two-pass algorithms have thus far shown better aggregate IO throughput.

[3]These are usually reported in Giga**bits** per second, instead of Giga**bytes** or Mega**bytes** per second, we have converted all to Megabytes per second to avoid confusion.

5.2 Equitable Quality

For our next experiment, we took several file types captured via VirusTotal, and used the ClarAvy tool [16] to determine benign and malicious labels. Not all file types are seen frequently or enough to build a production-quality model. This experiment is done purely to demonstrate that Zipf-Gramming produces models with similar final accuracy and that the results improve with the size of the dataset as the Zipf assumption holds. The five datasets and their AUC scores are shown in Table 2 are subsets of Ember3 before completion [17]. Each model was trained using Elastic-Net penalized logistic regression [26, 27, 36, 41, 60]. We see that Zipf-Gramming produces comparable feature sets but far faster, a requirement for timely updates to the production model.

Table 2: Five datasets ordered by total corpus size (i.e., number of n-grams equals the number of bytes). Deterministic calculation of top-k n-grams is nearly equal to Zipf-Gramming performance but computationally infeasible for regular model updating at $\approx 100\times$ slower. Zipf-Gramming's resulting models improve as the dataset size increases.

Dataset	Win32	Win64	ELF	.Net	PDF
Size	2.54 TB	1.85 TB	328 GB	189 GB	45 GB
Kilo-Gramming	95.73	96.48	99.29	96.48	82.98
Zipf-Gramming	95.44	96.37	99.26	94.63	81.59

5.3 Validating Top-k Accuracy

Having shown that our method is the fastest available for extracting the top-k most frequent *n*-grams, we next validate empirically that our theory is valid, and we obtain accurate estimates of the top-k items. This is distinct from showing accurate models, which is hypothetically possible with disjoint features. These experiments are carried out on real-world data, using a subset of Dot-Net benign and malicious programs totaling 188.93 GB in size as the maximum amount of data we could reasonably calculate the true top-k items. We perform this for 6-grams and 8-grams.

First, we look at the percentage of 6/8-grams missing from the top-k, and the relative error in the estimated count itself. For computational efficacy, this is measured by calculating the results for the top $k = 100,000$, and then using the ranked count order to evaluate all lower values of k. The results are shown below in Figure 3, where we can see that the percentage error is generally $\leq 2\%$.

The red and blue lines are the rates of missing grams from the top-k, which peaks at the lowest ranks of $k \leq 100$ due to errors in order and placement. This is primarily an artifact of using real-world data that are not truly I.I.D. distributed in the occurrence of n-grams, but the low error rate validates the theoretical assumption of Zipfian distributed features.

A key to the acceptability of the approximation by sub-sampling is that even if we make an error, the errors are inconsequential. For example, the $k + 1$'th most frequent item being selected instead of the k'th is unlikely to have an impact on model accuracy due to

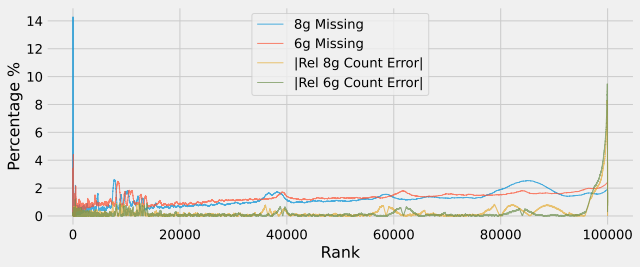

Figure 3: The percentage of missing top-k grams is small throughout the range of ranks (x-axis). The percent error in the estimated and true rank of an item in the k'th position is extremely small, usually $\leq 10^{-3}$%, and increases to a still inconsequential error rate at the very last k items.

similar occurrence rates. The yellow and green lines of Figure 3 show that the estimated count error of the 6/8-gram is very nearly 0 in almost all cases, with the error rate spiking only at the very bottom k ranks but at a range $\leq$ 10% indicating that these errors are inconsequential as we are selecting features with highly similar occurrence rates. If we had selected a problematic feature instead, such as one that was an order of magnitude further down the rank list, we would expect to see higher relative error rates.

Note in the high ranks, there are bursts of high variance caused by singleton outliers that are miss-ranked by large values, which occurs due to the non-I.I.D. distribution that occurs in real data. For example, the 6-gram 0x909090909090 is very common as the "No Operation" instruction (0x9) is repeated numerous times in a file for a variety of benign and malicious applications. By the nature of it's use, such occurrences are sequential in occurrence and thus clumped together instead of spread randomly throughout files. In such circumstances, our Zipfian approach is guaranteed to undercount the true occurrence rate of the feature. However, this does not happen with sufficient frequency to meaningfully change the results, and the mean error in rank is 11 and 16 for 6-grams and 8-grams, respectively.

5.4 Deployment Results

We now turn to the results of our method in deployment. Our previous model used byte 8-grams via the Kilo-Gramming algorithm for k = 1 million, and used Elastic-Net penalized logistic regression to build the model and down-select the number of relevant features to a more compact and deployable subset. This model was trained on approximately 2.5 million files from a proprietary corpus. Building this model required three to four days of compute time, and increasing the training set size had proven to be a cumbersome roadblock toward improvement. In internal testing, we observed that we could not simply replace all old data with new data either, as identifying old malware is still operationally useful.

Our new model was trained on an additional 2.6 million files, giving a total of 5.1 million files, and is 6.6 TB in size. Using the Zipf-Gramming approach, we extracted the top k = 10 million 6-grams, followed by Elastic-Net penalized logistic regression, which selected only 33 thousand features for the final model. *Despite our attempts to use this with the original Kilo-Gramming code, the estimated runtime to completion was 1 month, which was not viable fiscally or in terms of producing regularly updated models.* Developing the Zipf-gramming approach was what allowed us to produce new models of improved quality and with a runtime of just *four hours* on this corpus, is well within our ability to perform regular updates.

Multiple tests are used to evaluate our approach, each labeled using the aggregate results from Anti-Virus products in VirusTotal [1], using ClarAVy [16] to aggregate the reported output of each AV system to determine the benign/malicious label. These labels are high quality but can only be obtained months after the fact due to relying on AntiVirus outputs stabilizing. *We rely on them to retrospectively confirm our model's efficacy in a quantified way.*

First is a "Standard Test" set composed of data in a 3-month time span after the last observed data point in the training set. This is done to avoid test-set leakage due to correlations in time in the training data [34]. Second, we collect a "GAP" dataset that contains 132,700 samples collected 9 months after deployment, relevant to the Standard Test set, imposing a greater strain on the model. Third, we note that AV outputs change over time as they are updated, and we used a "Challenge" set that includes only malware sampled in an adversarial manner to be maximally difficult. Each Challenge file occurred after the original training data and was initially labeled as benign by all available AV products in VirusTotal when first seen, but was later labeled malicious by a plurality of AV products 30 days later. This indicates a sample of malware that is uniquely challenging because it was sufficiently different to be undetected when first deployed and only captured after the fact when AV systems made new updates. In this challenge set, there are 2533 Win32, 822 Win64, and 708 .Net files.

Table 3: Our new model performs better in our standard train/test split (first rows), on 9 months of proceeding production data (middle rows), and on a set of challenge malware that was missed by all Anti-Virus products (bottom rows).

	Standard Test		Gap			Challenge		
	F1	AUC	Win32	Win64	.Net	Win32	Win64	.Net
New	89.70	96.00	96.15	94.58	94.59	49.03	36.04	79.94
Old	73.43	91.35	92.02	64.43	97.60	32.45	0.81	0.14

Our results in Table 3 show that increasing the datasize massively improves performance, especially on challenging files that represent zero-day malware. This training size was not viable without improved feature extraction efficiency, and allows us to continue deploying a small (2MB) model that can run efficiently on many platforms via CPU only, as GPUs are not available or even compatible with many needed systems.

6 Conclusion

Byte n-grams are the only feature allowing us to satisfy a runtime/footprint critical use case for malware detection. By subsampling the data under the Zipf distribution assumption, we can carefully design a system thousands of times faster than Scikit-Learn, which enables us to produce a production-grade model in one day rather than a month as before. This improved our production model's identification of evasive challenging malware by up to 70% absolute points and was validated theoretically and empirically.

GenAI Usage Disclosure section

No generative AI of any kind was used at any point for this work.

References

[1] 2018. VirusTotal-Free online virus, malware and URL scanner. https://www.virustotal.com

[2] Milton Abramowitz. 1974. *Handbook of Mathematical Functions, With Formulas, Graphs, and Mathematical Tables,*. Dover Publications, Inc., USA.

[3] N. Balakrishnan. 1994. Order statistics from non-identical exponential random variables and some applications. *Computational Statistics & Data Analysis* 18, 2 (Sept. 1994), 203–238. doi:10.1016/0167-9473(94)90172-4

[4] Rebecca Schuller Borbely. 2015. On normalized compression distance and large malware. *Journal of Computer Virology and Hacking Techniques* (2015), 1–8. doi:10.1007/s11416-015-0260-0

[5] Marcus Botacin, Felipe Duarte Domingues, Fabrício Ceschin, Raphael Machnicki, Marco Antonio Zanata Alves, Paulo Lício de Geus, and André Grégio. 2021. AntiViruses under the Microscope: A Hands-On Perspective. *Computers & Security* i (Oct. 2021), 102500. doi:10.1016/j.cose.2021.102500

[6] Aaron Clauset, Cosma Rohilla Shalizi, and M E J Newman. 2009. Power-Law Distributions in Empirical Data. *SIAM Rev.* 51, 4 (2009), 661–703. doi:10.1137/070710111

[7] Ulrich Drepper. 2007. What every programmer should know about memory. *Red Hat, Inc* 11, 2007 (2007), 2007.

[8] Maksim E. Eren, Manish Bhattarai, Robert J. Joyce, Edward Raff, Charles Nicholas, and Boian S. Alexandrov. 2023. Semi-Supervised Classification of Malware Families Under Extreme Class Imbalance via Hierarchical Non-Negative Matrix Factorization with Automatic Model Selection. *ACM Trans. Priv. Secur.* 26, 4 (Nov. 2023), 48:1–48:27. doi:10.1145/3624567

[9] Agner Fog. 2022. Instruction tables: Lists of instruction latencies, throughputs and micro-operation breakdowns for Intel, AMD and VIA CPUs. *Technical University of Denmark* (2022).

[10] Daniel Gibert and Ramon Vicens. 2018. Classification of Malware by Using Structural Entropy on Convolutional Neural Networks. In *AAAI Conference on Artificial Intelligence*. 7759–7764.

[11] Siddhant Gupta, Fred Lu, Andrew Barlow, Edward Raff, Francis Ferraro, Cynthia Matuszek, Charles Nicholas, and James Holt. 2024. Living off the Analyst: Harvesting Features from Yara Rules for Malware Detection. In *2024 IEEE International Conference on Big Data (BigData)*. 2624–2634. doi:10.1109/BigData62323.2024.10825735 ISSN: 2573-2978.

[12] Olivier Henchiri and Nathalie Japkowicz. 2006. A Feature Selection and Evaluation Scheme for Computer Virus Detection. In *Proceedings of the Sixth International Conference on Data Mining*. IEEE Computer Society, Washington, DC, USA, 891–895. doi:10.1109/ICDM.2006.4 Series Title: ICDM '06.

[13] James Holt and Edward Raff. 2024. Malware Bytes. In *The Next Wave: Cyber Analytics Research*, Vol. 25. National Security Agency (NSA). Issue 1. https://www.govinfo.gov/app/details/GPO-TNW-25-1-2024/GPO-TNW-25-1-2024-6

[14] Jiyong Jang, David Brumley, and Shobha Venkataraman. 2011. BitShred: Feature Hashing Malware for Scalable Triage and Semantic Analysis. In *Proceedings of the 18th ACM conference on Computer and communications security - CCS*. ACM Press, New York, New York, USA, 309–320. doi:10.1145/2046707.2046742

[15] Robert J Joyce, Dev Amlani, Charles Nicholas, and Edward Raff. 2022. MOTIF: A Large Malware Reference Dataset with Ground Truth Family Labels. In *The AAAI-22 Workshop on Artificial Intelligence for Cyber Security (AICS)*. doi:10.48550/arXiv.2111.15031 arXiv: 2111.15031v1.

[16] Robert J. Joyce, Derek Everett, Maya Fuchs, Edward Raff, and James Holt. 2025. ClarAVy: A Tool for Scalable and Accurate Malware Family Labeling. In *Proceedings of the ACM Web Conference 2025 (WWW '25)*. Association for Computing Machinery, New York, NY, USA. doi:10.1145/3701716.3715212

[17] Robert J. Joyce, Gideon Miller, Phil Roth, Richard Zak, Elliott Zaresky-Williams, Hyrum Anderson, Edward Raff, and James Holt. 2025. EMBER2024 - A Benchmark Dataset for Holistic Evaluation of Malware Classifiers. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*. Association for Computing Machinery, New York, NY, USA, 5516–5526. doi:10.1145/3711896.3737431

[18] Robert J. Joyce, Tirth Patel, Charles Nicholas, and Edward Raff. 2023. AVScan2Vec: Feature Learning on Antivirus Scan Data for Production-Scale Malware Corpora. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. Association for Computing Machinery, New York, NY, USA, 185–196. doi:10.1145/3605764.3623907

[19] Robert J Joyce, Edward Raff, and Charles Nicholas. 2021. A Framework for Cluster and Classifier Evaluation in the Absence of Reference Labels. In *Proceedings of the 14th ACM Workshop on Artificial Intelligence and Security (AISec '21)*. Association for Computing Machinery. doi:10.1145/3474369.3486867 arXiv: 2109.11126v1.

[20] Robert J Joyce, Edward Raff, and Charles Nicholas. 2021. Rank-1 Similarity Matrix Decomposition For Modeling Changes in Antivirus Consensus Through Time. In *Proceedings of the Conference on Applied Machine Learning for Information Security*. arXiv: 2201.00757v1.

[21] Daniel Lemire. 2024. How fast is rolling Karp-Rabin hashing? https://lemire.me/blog/2024/02/04/how-fast-is-rolling-karp-rabin-hashing/. [Accessed 04-02-2025].

[22] Shijia Li, Jiang Ming, Pengda Qiu, Qiyuan Chen, Lanqing Liu, Huaifeng Bao, Qiang Wang, and Chunfu Jia. 2023. PackGenome: Automatically Generating Robust YARA Rules for Accurate Malware Packer Detection. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23)*. Association for Computing Machinery, New York, NY, USA, 3078–3092. doi:10.1145/3576915.3616625

[23] W. Li. 1992. Random texts exhibit Zipf's-law-like word frequency distribution. *IEEE Transactions on Information Theory* 38, 6 (1992), 1842–1845. doi:10.1109/18.165464

[24] Erik Linstead, Paul Rigor, Sushil Bajracharya, Cristina Lopes, and Pierre Baldi. 2007. Mining Internet-Scale Software Repositories. In *Advances in Neural Information Processing Systems*, Vol. 20. Curran Associates, Inc. https://papers.nips.cc/paper_files/paper/2007/hash/a532400ed62e772b9dc0b86f46e583ff-Abstract.html

[25] Chang Liu, Rebecca Saul, Yihao Sun, Edward Raff, Maya Fuchs, Townsend Southard Pantano, James Holt, and Kristopher Micinski. 2024. Assemblage: Automatic Binary Dataset Construction for Machine Learning. *Advances in Neural Information Processing Systems* 37 (Dec. 2024), 58698–58715. https://proceedings.neurips.cc/paper_files/paper/2024/hash/6bbefc73a187dd42e0dc065b4e7a0615-Abstract-Datasets_and_Benchmarks_Track.html

[26] Fred Lu, Ryan R. Curtin, Edward Raff, Francis Ferraro, and James Holt. 2024. High-Dimensional Distributed Sparse Classification with Scalable Communication-Efficient Global Updates. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)*. Association for Computing Machinery, New York, NY, USA, 2037–2047. doi:10.1145/3637528.3672038

[27] Fred Lu, Ryan R. Curtin, Edward Raff, Francis Ferraro, and James Holt. 2024. Optimizing the Optimal Weighted Average: Efficient Distributed Sparse Classification. arXiv:2406.01753 [cs.LG]

[28] Ahmed Metwally, Divyakant Agrawal, and Amr El Abbadi. 2005. Efficient Computation of Frequent and Top-k Elements in Data Streams. In *Proceedings of the 10th International Conference on Database Theory*. Springer-Verlag, Berlin, Heidelberg, 398–412. doi:10.1007/978-3-540-30570-5_27 Series Title: ICDT'05.

[29] M E J Newman. 2005. Power laws, Pareto distributions and Zipf's law. *Contemporary Physics* 46, 5 (2005), 323–351. doi:10.1080/00107510500052444 Publisher: Taylor & Francis.

[30] Andre T Nguyen, Edward Raff, and Aaron Sant-Miller. 2019. Would a File by Any Other Name Seem as Malicious?. In *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 1322–1331. doi:10.1109/BigData47090.2019.9006132

[31] Andre T Nguyen, Richard Zak, Luke E Richards, Maya Fuchs, Fred Lu, Robert Brandon, Gary Lopez Munoz, Edward Raff, Charles Nicholas, and James Holt. 2022. Minimizing Compute Costs: When Should We Run More Expensive Malware Analysis?, Vol. 3391. CEUR. https://ceur-ws.org/Vol-3391/paper6.pdf

[32] Tirth Patel, Fred Lu, Edward Raff, Charles Nicholas, Cynthia Matuszek, and James Holt. 2023. Small Effect Sizes in Malware Detection? Make Harder Train/Test Splits! *Proceedings of the Conference on Applied Machine Learning in Information Security* (2023).

[33] David B. Peizer and John W. Pratt. 1968. A Normal Approximation for Binomial, F, Beta, and other Common, Related Tail Probabilities, I. *J. Amer. Statist. Assoc.* 63, 324 (Dec. 1968), 1416–1456. doi:10.1080/01621459.1968.10480938

[34] Feargus Pendlebury, Fabio Pierazzi, Roberto Jordaney, Johannes Kinder, and Lorenzo Cavallaro. 2019. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time. In *28th USENIX Security Symposium (USENIX Security 19)*. USENIX Association, Santa Clara, CA, 729–746. https://www.usenix.org/conference/usenixsecurity19/presentation/pendlebury

[35] Steven T Piantadosi. 2014. Zipf's word frequency law in natural language: a critical review and future directions. *Psychonomic bulletin & review* 21, 5 (Oct. 2014), 1112–30. doi:10.3758/s13423-014-0585-6

[36] Edward Raff. 2017. JSAT: Java Statistical Analysis Tool, a Library for Machine Learning. *Journal of Machine Learning Research* 18, 23 (2017), 1–5. http://jmlr.org/papers/v18/16-131.html

[37] Edward Raff, William Fleming, Richard Zak, Hyrum Anderson, Bill Finlayson, Charles K. Nicholas, Mark Mclean, William Fleming, Charles K. Nicholas, Richard Zak, and Mark Mclean. 2019. KiloGrams: Very Large N-Grams for Malware Classification. In *Proceedings of KDD 2019 Workshop on Learning and Mining for Cybersecurity (LEMINCS'19)*. https://arxiv.org/abs/1908.00200

[38] Edward Raff and Mark McLean. 2018. Hash-Grams On Many-Cores and Skewed Distributions. In *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 158–165. doi:10.1109/BigData.2018.8622043

[39] Edward Raff and Charles Nicholas. 2018. Hash-Grams: Faster N-Gram Features for Classification and Malware Detection. In *Proceedings of the ACM Symposium on Document Engineering 2018*. ACM, Halifax, NS, Canada. doi:10.1145/3209280.3229085

[40] Edward Raff and Charles Nicholas. 2020. A Survey of Machine Learning Methods and Challenges for Windows Malware Classification. In *NeurIPS 2020 Workshop:*

ML Retrospectives, Surveys & Meta-Analyses (ML-RSA). arXiv:2006.09271 http://arxiv.org/abs/2006.09271 Best Paper Award.

[41] Edward Raff and Jared Sylvester. 2018. Linear models with many cores and cpus: A stochastic atomic update scheme. In *2018 IEEE International Conference on Big Data (Big Data)*. IEEE, 65–73.

[42] Edward Raff, Richard Zak, Russell Cox, Jared Sylvester, Paul Yacci, Rebecca Ward, Anna Tracy, Mark McLean, and Charles Nicholas. 2018. An investigation of byte n-gram features for malware classification. *Journal of Computer Virology and Hacking Techniques* 14 (2018), 1–20.

[43] Edward Raff, Richard Zak, Gary Lopez Munoz, William Fleming, Hyrum S. Anderson, Bobby Filar, Charles Nicholas, and James Holt. 2020. Automatic Yara Rule Generation Using Biclustering. In *13th ACM Workshop on Artificial Intelligence and Security (AISec'20)*. doi:10.1145/3411508.3421372 arXiv: 2009.03779.

[44] Philip H. Ramsey and Patricia P. Ramsey. 1988. Evaluating the Normal Approximation to the Binomial Test. *Journal of Educational Statistics* 13, 2 (June 1988), 173–182. doi:10.3102/10769986013002173

[45] S Redner. 1998. How popular is your paper? An empirical study of the citation distribution. *The European Physical Journal B - Condensed Matter and Complex Systems* 4, 2 (1998), 131–134. doi:10.1007/s100510050359

[46] Dominika Regéciová, Dušan Kolář, and Marek Milkovič. 2021. Pattern Matching in YARA: Improved Aho-Corasick Algorithm. *IEEE Access* 9 (2021), 62857–62866. doi:10.1109/ACCESS.2021.3074801 Conference Name: IEEE Access.

[47] Guillaume Rizk, Dominique Lavenier, and Rayan Chikhi. 2013. DSK: k-mer counting with very low memory usage. *Bioinformatics* 29, 5 (Jan. 2013), 652–653. doi:10.1093/bioinformatics/btt020

[48] Ethan M. Rudd, David Krisiloff, Scott Coull, Daniel Olszewski, Edward Raff, and James Holt. 2024. Efficient Malware Analysis Using Metric Embeddings. *Digital Threats* 5, 1 (March 2024), 4:1–4:20. doi:10.1145/3615669

[49] Ethan M Rudd, Mohammad Saidur Rahman, and Philip Tully. 2022. Transformers for End-to-End InfoSec Tasks: A Feasibility Study. In *Proceedings of the 1st Workshop on Robust Malware Analysis*. Association for Computing Machinery, New York, NY, USA, 21–31. doi:10.1145/3494110.3528242 Series Title: WoRMA '22.

[50] Jake Ryland Williams, Paul R Lessard, Suma Desu, Eric M Clark, James P Bagrow, Christopher M Danforth, and Peter Sheridan Dodds. 2015. Zipf's law holds for phrases, not words. *Scientific Reports* 5, 12209 (Aug. 2015). https://www.nature.com/articles/srep12209#supplementary-information Publisher: The Author(s).

[51] Diego Santoro, Leonardo Pellegrina, Matteo Comin, and Fabio Vandin. 2022. SPRISS: approximating frequentk-mers by sampling reads, and applications. *Bioinformatics* 38, 13 (May 2022), 3343–3350. doi:10.1093/bioinformatics/btac180

[52] Rebecca Saul, Chang Liu, Noah Fleischmann, Richard Zak, Kristopher Micinski, Edward Raff, and James Holt. 2024. Is Function Similarity Over-Engineered? Building a Benchmark. *Advances in Neural Information Processing Systems* 37 (Dec. 2024), 21636–21655. https://proceedings.neurips.cc/paper_files/paper/2024/hash/2663c994c84a79b338bca613fe1ae223-Abstract-Datasets_and_Benchmarks_Track.html

[53] Joshua Saxe and Konstantin Berlin. 2015. Deep neural network based malware detection using two dimensional binary program features. In *2015 10th International Conference on Malicious and Unwanted Software (MALWARE)*. IEEE, 11–20. doi:10.1109/MALWARE.2015.7413680 arXiv: 1508.03096.

[54] Ehsan Shareghi, Daniela Gerz, Ivan Vulić, and Anna Korhonen. 2019. Show Some Love to Your n-grams: A Bit of Progress and Stronger n-gram Language Modeling Baselines. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jill Burstein, Christy Doran, and Thamar Solorio (Eds.). Association for Computational Linguistics, Minneapolis, Minnesota, 4113–4118. doi:10.18653/v1/N19-1417

[55] Matthew Sills. 2024. Optimizing Rabin-Karp Hashing. https://mattsills.github.io/2024/03/02/rabin-karp/. [Accessed 04-02-2025].

[56] Guy L. Steele, Doug Lea, and Christine H. Flood. 2014. Fast splittable pseudorandom number generators. In *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications* (Portland, Oregon, USA) *(OOPSLA '14)*. Association for Computing Machinery, New York, NY, USA, 453–472. doi:10.1145/2660193.2660195

[57] Gil Tahan, Lior Rokach, and Yuval Shahar. 2012. Mal-ID: Automatic Malware Detection Using Common Segment Analysis and Meta-Features. *Journal of Machine Learning Research* 13, 33 (2012), 949–979. http://jmlr.org/papers/v13/tahan12a.html

[58] Daniele Ucci, Leonardo Aniello, and Roberto Baldoni. 2019. Survey of machine learning techniques for malware analysis. *Computers & Security* 81 (March 2019), 123–147. doi:10.1016/j.cose.2018.11.001

[59] Christian Wressnegger, Guido Schwenk, Daniel Arp, and Konrad Rieck. 2013. A close look on n-grams in intrusion detection. In *Proceedings of the 2013 ACM workshop on Artificial intelligence and security - AISec '13*. ACM Press, New York, New York, USA, 67–76. doi:10.1145/2517312.2517316

[60] Guo-xun Yuan, Chia-Hua Ho, and Chih-jen Lin. 2012. An improved GLMNET for L1-regularized logistic regression. *Journal of Machine Learning Research* 13 (2012), 1999–2030. doi:10.1145/2020408.2020421 Publisher: ACM Press Place: New York, New York, USA ISBN: 9781450308137.

[61] Richard Zak, Edward Raff, and Charles Nicholas. 2017. What can N-grams learn for malware detection?. In *2017 12th International Conference on Malicious and Unwanted Software (MALWARE)*. IEEE, 109–118. doi:10.1109/MALWARE.2017.8323963

[62] Haoyu Zhang, Qin Zhang, and Haixu Tang. 2018. Smooth q-Gram, and Its Applications to Detection of Overlaps Among Long, Error-Prone Sequencing Reads. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. ACM, New York, NY, USA, 267–276. doi:10.1145/3269206.3271688 Series Title: CIKM '18.

[63] George Kingsley Zipf. 1949. *Human behavior and the principle of least effort*. Addison-Wesley Press, Oxford, England. Publication Title: Human behavior and the principle of least effort..